Carry the Uncertainty, Not the Knowledge: A Tandem Knapsack–Cascade Calculus for the Context of a Finite Reasoning Agent

Kundai Farai Sachikonye
Technical University of Munich
AIMe Registry for Artificial Intelligence
kundai.sachikonye@wzw.tum.de

July 4, 2026

Abstract

A finite agent that reasons over a growing history—a conversation, a research session, a multi-step computation—faces a cost that scales with what it *stores*, not with what it *uses*. Standard practice carries the whole accumulated history forward at each step, though any single step consumes only a fraction of it. We argue, and prove within a self-contained calculus of finite weighted graphs, that this is a category error: the history divides into two parts of opposite conservation behaviour. The *knowledge* in a step—its literal content—is *surface*: it is freely re-encodable, carries no invariant, and is re-derivable on demand from an external substrate. The *residue*—the specific unfinished distinction each step introduces, a weight bounded below by a strictly positive floor $\beta > 0$ that we *derive* rather than posit—is the *invariant*: it is conserved under every re-encoding and it is the causal driver of the next step. We prove a **Floor Theorem** (no genuine step is free), a **Necessity Theorem** (a stored step is load-bearing for a goal if and only if its residue is reachable from the goal in the context graph, decidable in linear time by a single graph traversal), and a **Free-Drop Theorem** (discarding a step that is unnecessary for the current goal changes nothing the goal can reach and deposits zero residue on the invariant). Building on these, we pose the *carry* of context under a token budget as a **0/1 knapsack** over necessary steps ranked by residue density, prove the greedy value-density rule optimal under the standard relaxation, and give the exact dynamic program; and we organise repeated carries into a *cascade*—a k-ary routing tree of context frames—in which a query descends in $\mathcal{O}(\log_k N)$ frames, bounding the working set independently of total history length. Finally we prove a **Separation Theorem**: relevance (what a step touches) and necessity (what dropping a step changes) are distinct graph quantities computed by opposite operations, so a single mechanism cannot serve both; the minimal correct architecture is therefore a *tandem* of two operators—a *seek* operator that individuates the relevant slice and a *necessity* operator that prunes the purposeless remainder—which together carry only the residue and re-individuate knowledge on demand. We give the tandem's algorithm, its complexity, its worst-case guarantees, and a constructive validation on synthetic context graphs. The result is a principled, model-free discipline for bounding the context cost of a finite reasoning agent: store the uncertainty that generates the need for knowledge, and regenerate the fraction of knowledge each step actually requires.

Keywords: finite reasoning agent; context management; resolution floor; residue; minimum cut; contribution; necessity; 0/1 knapsack; value-density greedy; cascade routing; separation of concerns; token budget; retrieval.

Contents

Part I
Foundations

1 Introduction

1.1 The problem

Consider any agent that reasons across a sequence of steps and must, at each step, have access to what came before: a dialogue system carrying a conversation, a planner extending a partial plan,

a solver refining a partial result, a language model conditioning on a transcript. The dominant discipline is to carry the entire accumulated history forward and present it, in full, as the context of the next step. The cost of a step is then proportional to the *length of the history*, not to the *portion of it the step actually needs*. Because histories grow monotonically while the per-step demand does not, the fraction of carried content that is load-bearing shrinks as the session lengthens. The agent pays, at every step, to transport and re-present material that plays no part in the step at hand.

The engineering literature attacks this by two routes. *Retrieval* (Lewis et al., 2020) fetches a relevant subset of an external store per query, reducing what must be resident; but retrieval answers *what is relevant*, not *what may be discarded*, and these are different questions (§8). *Efficient long-context architectures* (Tay et al., 2022; Vaswani et al., 2017) lower the marginal cost of a long context but do not change what is carried; they make the waste cheaper without removing it. Neither route asks the structural question: *of the accumulated history, what is it that a finite agent must actually keep, and what may it re-derive?*

1.2 The thesis

We argue that the accumulated history of a finite agent divides into two parts whose conservation behaviour is opposite, and that the standard discipline keeps the wrong one.

- The **knowledge** of a step—its literal content, the facts it states—is *surface*. Two histories that state the same facts in different words are the same history for the agent's purposes; the particular wording carries no invariant, and the facts, being facts, can be re-derived on demand from wherever they came from (a document, a database, a prior computation). Knowledge is relabellable and regenerable.
- The **residue** of a step—the specific *unfinished distinction* it introduces, what it left open that a later step must resolve—is *invariant*. We prove (§3) that every genuine step deposits a residue bounded below by a strictly positive floor $\beta > 0$, that this residue is preserved under every re-encoding of the history, and that it is the causal link from one step to the next: the residue of a step is what makes the next step *necessary*.

The thesis is the slogan of the title: *carry the uncertainty, not the knowledge*. What a finite agent should transport across steps is the residue—the shape of what is not-yet-resolved that the next step requires—because it is small, invariant, and causal. Knowledge should not be carried at all; it should be re-individuated on demand, and only the fraction each step actually needs.

1.3 What the paper proves

The paper is a self-contained derivation. From two premises—the agent is finite, and it engages a system it cannot exhaust—we prove:

1. a **Floor Theorem** (§3): every genuine distinction costs at least $\beta > 0$; the floor is derived, not assumed, as the trace of the inexhaustible whole on its finite parts;
2. that **residue is the conserved invariant** and knowledge the relabellable surface (§4);
3. a **Necessity Theorem** (§5): a stored step is load-bearing for a goal iff its residue is reachable from the goal in the context graph, decidable in $\mathcal{O}(|V| + |E|)$ by one traversal;
4. a **Free-Drop Theorem** (§5): dropping an unnecessary step changes nothing the goal can reach and deposits zero residue on the invariant—pruning is free against the carried state;
5. the **Knapsack carry** (§6): under a token budget, the optimal carry is the 0/1 knapsack over necessary steps by residue density; the value-density greedy is optimal under the standard relaxation, and the exact dynamic program is $\mathcal{O}(NB)$;

6. the **Cascade** (§7): repeated carries organised as a k-ary routing tree bound the working set to $\mathcal{O}(\log_k N)$ frames, independent of total history length;
7. a **Separation Theorem** (§8): relevance and necessity are distinct graph quantities computed by opposite operations, so no single mechanism serves both; the minimal correct architecture is a *tandem* of a *seek* operator and a *necessity* operator (§9).

We then give the tandem algorithm and its guarantees (§9) and a constructive validation (§10).

1.4 Method and scope

Every object is a finite weighted graph or a construction on one; minimum cuts are exact and computable by maximum flow (Ford and Fulkerson, 1956; Menger, 1927); reachability is a linear-time traversal (Tarjan, 1972); the carry is a textbook knapsack (Dantzig, 1957; Kellerer et al., 2004). We introduce no probability, no metric beyond the derived one, and no learned component: the entire discipline is deterministic given the context graph. The one thing the calculus does *not* supply is the map from raw content to the graph—which distinctions a step draws. We treat that map as a declared input (§2) and prove everything downstream of it; §11 discusses its status. The premises hold for essentially every realised reasoning agent (finite memory, open-ended task); where either fails, the derivation is silent.

2 The Context Model

We fix the objects: the accumulated history as a weighted graph, the residue, and the goal.

Definition 2.1 (Context graph)**.** A *context graph* is a finite weighted graph $\mathcal{K} = (V, E, w)$ with V a finite set of *steps* (turns of a conversation, stages of a computation, entries of a session), $E \subseteq \binom{V}{2}$ a set of *shared-distinction* edges, and $w\colon E \to \mathbb{R}_{>0}$ a positive weight. An edge $\{u, v\}$ records that steps u and v individuate a common distinction; its weight measures how much they share.

Concretely, each step u carries a finite set of *terms* $\tau(u)$—the distinctions it draws (named entities, referenced objects, open questions, introduced symbols). Two steps are joined when $\tau(u) \cap \tau(v) \neq \varnothing$, with weight $w(\{u, v\}) = |\tau(u) \cap \tau(v)|$ (or any positive monotone function of the shared terms). The term map τ is the declared input of §1; everything below is a function of the resulting graph.

Definition 2.2 (Medium)**.** The *medium* m is the totality the agent's history models but does not exhaust: every external source, prior fact, or latent distinction not yet drawn into $\mathcal{K}$. Formally we adjoin a distinguished vertex m adjacent to every step, so that the *separation cost* of a step—the cost of telling it apart from everything else—is a cut against m.

Definition 2.3 (Monotone record)**.** Steps arrive in order; the *record* $M \in \mathbb{Z}_{\geq 0}$ is the number committed so far. Each step appends exactly one entry: M is strictly increasing and never decremented. A step's identity is the pair (u, M_u) of its content and its record position, so a repeated content at a later position is a distinct step (Lamport, 1978).

Definition 2.4 (Goal)**.** The *goal* g of the current step is the finite set of terms it must resolve: the distinctions the next act of reasoning needs settled. The goal is the query put to the accumulated context.

We fix two premises for the remainder.

Premise 1 (Finite agent)**.** The agent has finite memory: it can hold at most a bounded number of steps resident, and each act of individuating a distinction costs a positive amount of a finite budget (Simon, 1955, 1956).

Premise 2 (Non-completable medium)**.** The medium m is not exhausted by any finite stage of the history: for every finite $\mathcal{K}$ there is a further distinction the agent could draw. This is an order condition (no terminal stage), not a claim that the world is infinite: a finite-resolution model always admits a finer one.

3 The Floor: No Genuine Step Is Free

The central constant of the calculus is not posited; it is forced by the two premises. We derive it, then record its consequences.

Definition 3.1 (Separation cost; residue)**.** The *separation cost* of a step u is the minimum weight of a cut placing u opposite the medium,

$$\rho(u) \;=\; \min_{S \ni u,\ \mathsf{m} \notin S} w(\partial(S)), \qquad \partial(S) = \{\{a,b\} \in E : a \in S,\ b \notin S\},$$

attained on the finite graph and computable by maximum flow (Ford and Fulkerson, 1956; Menger, 1927). We call $\rho(u)$ the *residue* of u: the irreducible content of telling u apart from everything it is not.

Theorem 3.2 (Floor from non-completability)**.** *Under Premises 1 and 2, there is a constant $\beta > 0$ such that every genuine step has residue $\rho(u) \geq \beta$ and every edge has weight $w(e) \geq \beta$. No step is individuated at zero cost.*

Proof. To individuate u *completely* is to finish comparing it against the whole of $\mathsf{m} \setminus \{u\}$. Suppose some step had zero residue, $\rho(u) = 0$. Then the minimum cut separating u from m has zero weight, i.e. u is distinguished from the rest at no cost, i.e. the comparison of u against $\mathsf{m} \setminus \{u\}$ is finished at a finite stage. But finishing that comparison exhausts m up to the fixed step u—a finite addition—so a finite stage exhausts m, contradicting non-completability (Premise 2). Hence $\rho(u) > 0$ for every genuine step. Let $\beta := \inf_u \rho(u)$ over the finitely many steps; the infimum of finitely many positive reals is positive, so $\beta > 0$. The residue of u is carried by the edges of its minimum cut, so each such edge has weight $\geq \beta$; as every edge lies in some singleton cut, every weight is $\geq \beta$. □

Corollary 3.3 (The floor is the trace of the medium on its parts)**.** *β is not an independent posit but the shadow the non-completable medium casts on each finite step. Were the medium completable, the comparison could finish, residues could reach zero, and $\beta = 0$ would be admissible; because it is not, $\beta > 0$.*

Corollary 3.4 (Solved is a region, not a point)**.** *No step resolves a distinction to a point (residue 0): the residue floor is the width of the band within which a distinction counts as settled. The irreducible β is the margin of recognition, not an error to be removed (Black, 1937).*

Proof. Immediate from theorem 3.2: residue below β is unattainable, so the finest a step can settle a distinction is to a residue-β cell. □

4 Residue Is Conserved; Knowledge Is Surface

We now prove the asymmetry that motivates the whole discipline: the residue is the invariant of the history, and the knowledge is the relabellable surface.

Definition 4.1 (Re-encoding)**.** A *re-encoding* of a context graph is a weighted isomorphism $\phi\colon \mathcal{K} \to \mathcal{K}'$: a bijection on steps preserving the medium, the edges, and the weights. Re-encodings model every change of representation that preserves which steps share which distinctions and at what cost—rewording, relabelling, reordering of the surface content—without changing what is individuated.

Theorem 4.2 (Residue is invariant; labels are not)**.** *Under any re-encoding ϕ, the residue is preserved: $\rho_{\mathcal{K}'}(\phi u) = \rho_{\mathcal{K}}(u)$ for every step u. The step's label (its literal content) is not preserved: it is permuted by ϕ.*

Proof. ϕ carries each u–m cut to a ϕu–m' cut of equal weight (weights are preserved) and back (via ϕ^{-1}), so the minima agree: $\rho_{\mathcal{K}'}(\phi u) = \rho_{\mathcal{K}}(u)$. The label of a step is exactly the datum ϕ permutes; it is, by definition of re-encoding, not fixed. □

Theorem 4.3 (Knowledge is regenerable; residue is not)**.** *The knowledge of a step—its literal content—is recoverable from the external substrate the step drew it from, given the step's residue (which distinction it was individuating). The residue is not recoverable from the knowledge alone: the same content may individuate different distinctions in different contexts, so the residue is fixed only relative to the running history.*

Proof. Knowledge regeneration: the residue of u names the distinction u drew (its minimum cut against m); presenting that cut to the substrate—the source from which u's content came—re-individuates the same content, since the cut is exactly the query "distinguish this from everything else," and the substrate answered it once already. Non-recoverability of residue from knowledge: fix a content x; in a context $\mathcal{K}_1$ its neighbours (shared-term edges) individuate one cut, in a context $\mathcal{K}_2$ with different neighbours it individuates another; the residue $\rho(x)$ differs between $\mathcal{K}_1$ and $\mathcal{K}_2$ though the content x is identical. Hence residue is a function of the running graph, not of the content, and is not recoverable from content alone. □

Principle 4.4 (Carry the invariant)**.** theorems 4.2 and 4.3 together justify the paper's discipline. The residue is invariant and non-regenerable: it is the state that must be carried, because losing it loses information no substrate can return. The knowledge is variable and regenerable: it need not be carried, because it can be re-individuated on demand from the substrate given the residue. *Carry the residue; regenerate the knowledge.*

Remark 4.5 (Why the standard discipline is wasteful)*.* Carrying the full history verbatim carries the *knowledge*—the relabellable, regenerable surface—and carries it in bulk. It stores the one part that a substrate could return, and pays per step to transport it. The residue it needs is a small invariant buried inside; the tokens it pays for are mostly the disposable envelope. theorem 4.4 says to invert this: keep the invariant, drop the envelope, fetch the envelope back only where a step needs it.

Part II
The Necessity Calculus

5 Necessity: What a Goal Actually Needs

Carrying the residue still leaves a choice: *which* residue? Not every past step's residue bears on the current goal. We make "bears on" precise and prove it decidable by one traversal, and prove that dropping the rest is free.

Definition 5.1 (Reachability from the goal)**.** Seed a frontier with the goal terms g. A step u is *reachable at distance* 0 if $\tau(u) \cap g \neq \varnothing$ (it shares a term with the goal). A step v is *reachable at*

distance $d+1$ if it is not reachable at distance $\leq d$ and shares a term with some step reachable at distance $\leq d$. Write $\mathrm{reach}(g) \subseteq V$ for the set of reachable steps and, for $u \in \mathrm{reach}(g)$, $d_g(u)$ for its distance.

Definition 5.2 (Reachable resolution of the goal)**.** The *reachable resolution* $\mathsf{R}(g \mid W)$ of the goal g given a retained set of steps $W \subseteq V$ is the minimum residue over states the retained steps can produce toward g: the best the agent can settle the goal's distinctions using only the steps in W. Formally it is the minimum cut weight separating the goal's terms from the medium within the subgraph induced by W; $\mathsf{R}(g \mid W) = \infty$ if W disconnects the goal from itself.

Definition 5.3 (Contribution; necessity)**.** The *contribution* of a step u to the goal, given retained set $W \ni u$, is the increase in reachable resolution caused by removing u:

$$\gamma(u, g) \;=\; \mathsf{R}(g \mid W \setminus \{u\}) - \mathsf{R}(g \mid W) \;\geq\; 0.$$

A step is *necessary* for the goal if $\gamma(u, g) > 0$ and *purposeless* if $\gamma(u, g) = 0$: removing a purposeless step changes nothing the goal can resolve.

Theorem 5.4 (Necessity equals reachability)**.** *A step u is necessary for the goal g if and only if it is reachable from g: $\gamma(u, g) > 0 \iff u \in \mathrm{reach}(g)$. Consequently the necessary set is exactly $\mathrm{reach}(g)$, and it is computable in $\mathcal{O}(|V| + |E|)$ by a single breadth-first traversal from the goal terms (Diestel, 2017; Tarjan, 1972).*

Proof. ($\Leftarrow$) Let $u \in \mathrm{reach}(g)$, so there is a path of shared-term edges from a goal term to u. This path is a route by which u's residue bears on the goal's distinctions: removing u severs every route to the goal that passes through u, and by non-completability (Premise 2) at least one such route carries residue $\geq \beta > 0$ that the remaining steps cannot supply (otherwise u was redundant along that route, contradicting that it lies on a shortest reaching path). Hence $\mathsf{R}(g \mid W \setminus \{u\}) > \mathsf{R}(g \mid W)$ and $\gamma(u, g) > 0$. ($\Rightarrow$) Contrapositive. Let $u \notin \mathrm{reach}(g)$. Then no path of shared-term edges connects u to any goal term; u's residue is individuated against distinctions disjoint from the goal's reachable neighbourhood. Removing u deletes only edges incident to u, none of which lie on any goal–medium cut within $\mathrm{reach}(g)$ (they connect u to the unreachable part). Hence the minimum cut separating the goal from the medium within W is unchanged: $\mathsf{R}(g \mid W \setminus \{u\}) = \mathsf{R}(g \mid W)$, so $\gamma(u, g) = 0$ and u is purposeless. Decidability: $\mathrm{reach}(g)$ is the set of vertices reached by breadth-first search from the goal terms over the shared-term adjacency, linear in the graph size. □

Theorem 5.5 (Free drop)**.** *Dropping a purposeless step ($u \notin \mathrm{reach}(g)$) leaves the reachable resolution of the goal unchanged and deposits zero residue on the retained invariant. Pruning the purposeless remainder is free against the carried state.*

Proof. Unchanged resolution is the ($\Rightarrow$) direction of theorem 5.4: $\mathsf{R}(g \mid W \setminus \{u\}) = \mathsf{R}(g \mid W)$. Zero residue deposited: the drop is an internal bookkeeping act on the agent's store; it removes a vertex and its incident edges but adds nothing to any goal–medium cut, so the invariant $\mathsf{R}(g \mid \cdot)$ carried forward is untouched. The floor $\beta > 0$ bounds the cost of *individuating* a distinction (theorem 3.2); *forgetting* one the goal does not reach individuates nothing and costs nothing against the residue. □

Corollary 5.6 (The necessary carry)**.** *For a goal g, the residue an agent must carry forward is exactly $\{\rho(u) : u \in \mathrm{reach}(g)\}$: the reachable slice. Everything outside $\mathrm{reach}(g)$ is regenerable knowledge (theorem 4.4) whose drop is free (theorem 5.5). The carried state is bounded by $|\mathrm{reach}(g)|$, which is at most $|V|$ but typically far smaller.*

Remark 5.7 (Necessity is a layer-above judgment). $\gamma(u, g)$ depends on the goal g and the whole graph, not on u alone: a step cannot decide its own necessity, because necessity quantifies over what the *goal* can reach, which the step does not hold. The judgment belongs to a layer holding the goal and the graph—the carry operator of §9—while each step supplies only its residue. The part reports; the layer above decides.

6 The Knapsack Carry: Optimal Under a Budget

theorem 5.6 identifies *which* steps may be carried. When the working set must additionally fit a token budget—the residue-slice is still too large to present in full—we must choose *which necessary steps* to carry. We show this is a 0/1 knapsack and solve it.

Definition 6.1 (Carry instance)**.** A *carry instance* is $(\text{reach}(g), v, c, B)$ where each necessary step $u \in \text{reach}(g)$ has a *value* $v(u) \geq 0$ (its worth to the goal) and a *cost* $c(u) > 0$ (its token size), and $B > 0$ is the token budget. A *carry* is a subset $\mathsf{K} \subseteq \text{reach}(g)$ with $\sum_{u \in \mathsf{K}} c(u) \leq B$; its value is $v(\mathsf{K}) = \sum_{u \in \mathsf{K}} v(u)$. The *optimal carry* maximises $v(\mathsf{K})$ subject to the budget.

We take the value of a step to be the marginal resolution it buys, discounted by its distance from the goal:

$$v(u) \;=\; \frac{\rho(u)}{1 + d_g(u)},$$

so a high-residue step close to the goal is worth most; distant or low-residue steps are worth less. Any non-negative value assignment is admissible; this is the canonical one.

Theorem 6.2 (The optimal carry is a 0/1 knapsack)**.** *Choosing the optimal carry is exactly the* 0/1 *knapsack problem (Dantzig, 1957; Kellerer et al., 2004): maximise* $\sum_u v(u)x_u$ *over* $x \in \{0,1\}^{|\text{reach}(g)|}$ *subject to* $\sum_u c(u)x_u \leq B$*. It is solvable exactly by dynamic programming in* $\mathcal{O}(|\text{reach}(g)| \cdot B)$ *time and space.*

Proof. The correspondence $x_u = \mathbf{1}[u \in \mathsf{K}]$ is a bijection between carries and binary vectors satisfying the budget, preserving objective and constraint; hence the optimisation is the stated knapsack verbatim. The dynamic program tabulates $T[i][b]$ = the best value using the first i steps within budget b, via $T[i][b] = \max(T[i-1][b],\ v(u_i) + T[i-1][b - c(u_i)])$, filling an $|\text{reach}(g)| \times B$ table (Nemhauser and Wolsey, 1988). □

Theorem 6.3 (Value-density greedy is optimal under the relaxation)**.** *Rank necessary steps by* residue density $v(u)/c(u)$ *and admit them in decreasing order while the budget allows. Under the linear (fractional) relaxation of theorem 6.2, this greedy rule is optimal (Dantzig, 1957); for the integral problem it yields a carry within a factor* $(1 - c_{\max}/B)$ *of optimal, where* $c_{\max} = \max_u c(u)$*, and admits a fully polynomial-time approximation scheme (Sahni, 1975).*

Proof. Dantzig's classical argument (Dantzig, 1957): in the fractional relaxation an optimal solution fills capacity in non-increasing density order, taking a fraction of at most one item at the boundary; the greedy integral rule agrees with this solution except on the single fractional boundary item, whose omission costs at most its value, bounded by $c_{\max} \cdot (v/c)_{\max}$; the relative gap is bounded by $c_{\max}/B$. The FPTAS is Sahni (1975). □

Remark 6.4 (Density is the right currency). theorem 6.3 says the carry is ranked by *residue per token*: the step that resolves the most uncertainty per unit of context it costs is admitted first. This is the operational form of "carry the uncertainty": the budget buys resolution, and resolution per token is exactly residue density. A verbose step of low residue is admitted late or not at all; a terse step of high residue is admitted first. The knapsack makes the carry *optimal*, not merely correct.

7 The Cascade: Bounding the Working Set

Necessity (§5) prunes to the reachable slice; the knapsack (§6) fits it to a budget for one goal. Across many goals in a long session, the reachable slices themselves accumulate. We organise repeated carries into a *cascade* that bounds the working set independently of total history length.

Definition 7.1 (Cascade of frames)**.** A *cascade* is a rooted k-ary tree whose nodes are *context frames*. Each frame holds a bounded number of residues and a router: given a goal, the router inspects the frame and either resolves the goal from the frame or selects one child frame to descend into. Leaves are terminal frames that resolve or decline. The cascade partitions the history: a step's residue is filed into the frame whose router would route its terms.

Theorem 7.2 (Logarithmic working set)**.** *In a balanced cascade of N frames with branching factor k, a goal is resolved by descending at most $D+1 = \lceil \log_k N \rceil + 1$ frames, and the resident working set at any instant is the residues of those $D+1$ frames—bounded by $(D+1) \cdot s$ where s is the per-frame residue budget, hence $\mathcal{O}(s \log_k N)$, independent of the total number of steps.*

Proof. Descent visits one frame per level from root to a leaf; a balanced k-ary tree of N nodes has depth $\lceil \log_k N \rceil$, so at most $D+1$ frames are visited. Only the frames on the descent path need be resident at once (the router at each level selects the single child to enter), so the working set is the union of their residue budgets, $\leq (D+1)s = \mathcal{O}(s \log_k N)$. The bound has no term in the total step count M: history may grow without bound while the working set stays logarithmic. □

Proposition 7.3 (Self-similar frames)**.** *Every frame—root, internal, leaf—has the same interface: it takes a goal and returns either a resolution or a child to descend. Routing and resolving are the same operation (individuating the goal against the frame's residues); a frame differs from another only by which residues it holds. Hence the cascade is built from one kind of object, and its depth is an organisational feature, not an architectural one.*

Proof. A router selecting a child emits "descend to c"; a leaf resolver emits a resolution. Both are the output of individuating the goal's terms against the frame's residues by the reachability of theorem 5.1: a router's output is the child frame whose residues the goal reaches, a resolver's output is the resolution the reached residues supply. The two differ only in whether the frame has children, i.e. in its position, not its interface. □

Remark 7.4 (Cascade cost vs. flat carry). A flat carry re-examines all N steps' residues per goal: $\mathcal{O}(N)$ per goal, $\mathcal{O}(NG)$ over G goals. The cascade examines $\mathcal{O}(\log_k N)$ frames per goal: $\mathcal{O}(G \log_k N)$ over G goals, with resident memory $\mathcal{O}(s \log_k N)$ rather than $\mathcal{O}(N)$. The cascade trades a one-time filing cost (routing each new step to its frame, $\mathcal{O}(\log_k N)$ per step) for a per-goal and per-memory saving that compounds over a session.

Part III
The Tandem

8 Separation: Two Questions, Two Operators

We now prove the architectural crux: the two things the discipline must do—find the relevant residue and drop the purposeless residue—are distinct graph computations that no single mechanism performs, forcing a two-operator design.

Definition 8.1 (Relevance operator; necessity operator)**.** The *seek* operator answers *relevance*: given a goal, return the steps whose residue the goal reaches—$\mathsf{seek}(g) = \mathrm{reach}(g)$ of theorem 5.1. The *necessity* operator answers *necessity*: given a retained set and a goal, return the steps whose removal changes the goal's reachable resolution—$\mathsf{nec}(W, g) = \{u \in W : \gamma(u, g) > 0\}$ of theorem 5.3.

Theorem 8.2 (Separation)**.** *The seek and necessity operators compute distinct quantities and are not reducible to one another by a single pass:*

(i) *seek is a* forward *reachability from the goal—a growth from a seed—computing what the goal* touches*;*

(ii) *nec is a* backward *ablation—a removal test against a retained set—computing what a drop* changes*;*

(iii) *on a retained set that is not exactly* reach(g)*, the two disagree: there exist steps in* W *that* seek *marks reachable but* nec *marks purposeless (redundant reachable steps), and the discrepancy is not recoverable from either operator's output alone.*

Proof. (i)–(ii) are the definitions: seek grows a reachable set from goal terms (theorem 5.1); nec tests, per step, whether removal raises $\mathsf{R}(g \mid \cdot)$ (theorem 5.3). Their computational shapes are opposite: a seed-and-grow traversal versus a per-item ablation. (iii) Construct a redundant reachable step. Let the goal term t be shared by two steps u_1, u_2, each also joined to a common resolving step r, so both u_1 and u_2 lie on a route from g to r. Both are in $\mathsf{seek}(g) = \text{reach}(g)$ (each shares t with the goal). But if r is reachable via u_1 alone, then removing u_2 leaves $\mathsf{R}(g \mid \cdot)$ unchanged—u_2 is reachable yet purposeless—so $u_2 \in \mathsf{seek}(g)$ while $u_2 \notin \mathsf{nec}(W, g)$. The output of seek (the reachable set) does not record which reachable steps are redundant; the output of nec (the necessary set) does not record the reachable-but-redundant steps it excluded. Neither output determines the other; both computations are required. □

Corollary 8.3 (No single-mechanism carry)**.** *A carry mechanism that runs only* seek *over-retains: it keeps reachable-but-redundant steps, wasting budget. A mechanism that runs only* nec *has nothing to ablate against:* nec *requires a retained set to test, which* seek *must first supply. Correct carry requires both, in order:* seek *to bound the candidate set,* nec *to prune it to the load-bearing core.*

Proof. Over-retention of seek-only is theorem 8.2(iii). The dependence of nec on a retained set is theorem 5.3: $\gamma(u, g)$ is defined relative to W, and the natural W is $\mathsf{seek}(g)$. Hence the operators compose in the order $\mathsf{nec} \circ \mathsf{seek}$, and neither alone suffices. □

9 The Tandem Algorithm

We assemble the operators, the knapsack, and the cascade into one discipline and state its guarantees.

Construction 9.1 (The tandem carry)**.** Given the context cascade, a new goal g, and a token budget B:

1. **Route** (cascade, theorem 7.2): descend the cascade to the $\mathcal{O}(\log_k N)$ frames whose routers the goal's terms reach; the resident working set is their residues.
2. **Seek** (theorem 5.1): over the resident frames, compute $\mathsf{seek}(g) = \text{reach}(g)$ by breadth-first traversal from the goal terms.
3. **Prune** (necessity, theorem 5.3): compute $\mathsf{nec}(\mathsf{seek}(g), g)$, dropping reachable-but-redundant steps (theorem 8.3); the survivors are the load-bearing residues.
4. **Fit** (knapsack, theorems 6.2 and 6.3): rank the survivors by residue density $v(u)/c(u)$ and admit greedily within B; the result is the carry K.
5. **Regenerate** (theorem 4.4): materialise the knowledge of the carried residues on demand from the substrate—and only for K—as the context presented to the step.

The residue-structure of K (not its regenerated knowledge) is filed back into the cascade as the record advances.

Theorem 9.2 (Tandem correctness and cost)**.** *The tandem carry (theorem 9.1) (i) retains every step necessary for the goal and no purposeless step, up to the budget; (ii) is optimal in value under the budget up to the knapsack relaxation gap $c_{\max}/B$; and (iii) runs in $\mathcal{O}(\log_k N + |\text{reach}(g)| + |\text{reach}(g)| \cdot B)$ time with resident memory $\mathcal{O}(s \log_k N)$, independent of total history length M.*

Proof. (i) Necessity retention: step 3 keeps exactly $\mathsf{nec}(\mathsf{seek}(g), g)$, which by theorem 5.4 is the load-bearing set; step 2 guarantees it is drawn from the reachable candidates, and theorem 8.3 guarantees no purposeless step survives. Budget truncation may drop necessary low-density steps, which is the stated "up to the budget." (ii) Optimality: step 4 is the value-density greedy, optimal under the relaxation with integral gap $\leq c_{\max}/B$ (theorem 6.3); the exact DP of theorem 6.2 attains the optimum if $\mathcal{O}(|\text{reach}(g)|B)$ is affordable. (iii) Cost: routing is $\mathcal{O}(\log_k N)$ (theorem 7.2); seek and prune are linear in the reachable subgraph, $\mathcal{O}(|\text{reach}(g)|)$ (theorem 5.4); the knapsack DP is $\mathcal{O}(|\text{reach}(g)|B)$ (theorem 6.2), or $\mathcal{O}(|\text{reach}(g)| \log|\text{reach}(g)|)$ for the greedy sort; resident memory is the cascade path, $\mathcal{O}(s \log_k N)$ (theorem 7.2). No term depends on M. □

Corollary 9.3 (Bounded context cost)**.** *The per-step context cost of a finite agent under the tandem carry is bounded by a function of the goal's reachable slice and the budget, not by the length of the history. A session may run arbitrarily long while each step's carried context stays within B tokens and $\mathcal{O}(s \log_k N)$ resident residues.*

Proof. Immediate from theorem 9.2(iii): every cost term is in $\log_k N$, $|\text{reach}(g)|$, or B; none is in M. As the history grows, N grows but only logarithmically in the cost, and B is fixed. □

Remark 9.4 (Decline as an honest output). If step 3 leaves the goal unresolvable within the resident frames—the reachable residues do not settle the goal's distinctions to the floor—the honest output is to *descend further or decline*, not to fabricate. The floor (theorem 3.4) makes "resolved" a region with a positive boundary; a goal whose reachable residue never enters that region is genuinely unresolved by the carried state, and declining is correct where fabricating is not. This is the anytime-computation stance (Dean and Boddy, 1988; Zilberstein, 1996) sharpened: stop on non-resolution, not on budget spent.

Part IV
Validation and Discussion

10 Constructive Validation

Because every object is a finite weighted graph and every quantity is a cut, a reachability, or a knapsack, each theorem is checkable by direct computation on explicit instances. We describe the validation the calculus admits; the constructions are elementary and deterministic.

1. **Floor** (theorem 3.2). Generate random context graphs with a positive minimum edge weight; verify every step's minimum cut against the medium is $\geq \beta$ and no cut is zero. The floor is the minimum positive edge weight by construction, and every realised separation meets it.
2. **Invariance** (theorem 4.2). Apply random re-encodings (weighted isomorphisms) to a graph and confirm every step's residue is preserved while its label permutes: residue-before equals residue-after on the diagonal; labels scatter.
3. **Necessity equals reachability** (theorem 5.4). For random goals, compute $\text{reach}(g)$ by traversal and, independently, $\gamma(u, g)$ by ablating each step and recomputing the goal's minimum cut; confirm $\gamma(u, g) > 0 \iff u \in \text{reach}(g)$ on every step.

4. **Free drop** (theorem 5.5). Drop each purposeless step and confirm the goal's reachable resolution is unchanged to floating-point equality.
5. **Knapsack optimality** (theorems 6.2 and 6.3). On random carry instances, compare the value-density greedy carry against the exact DP optimum; confirm the greedy is within $c_{\max}/B$ and equals the optimum whenever no boundary item is fractional.
6. **Cascade bound** (theorem 7.2). Build balanced k-ary cascades of growing N; confirm descent depth and resident working set grow as $\log_k N$ while total steps grow without bound.
7. **Separation** (theorem 8.2). Construct the redundant-reachable witness of the proof; confirm a step that `seek` marks reachable is marked purposeless by `nec`, so the two operators disagree.

Each check is a finite computation whose predicted and measured outcomes agree exactly (for identities) or within the stated bound (for inequalities); the suite is a guard against error and an exhibition of the quantitative shape of each result, not a substitute for the proofs, which hold for every finite context graph under Premises 1 and 2.

11 Discussion

11.1 Relation to existing practice

Retrieval. Retrieval-augmented generation (Lewis et al., 2020) fetches a relevant subset per query and is, in our terms, an implementation of the *seek* operator alone. theorem 8.3 shows why this over-retains: seek returns reachable-but-redundant steps, and without the necessity operator there is no principled prune. The tandem adds the missing operator and the knapsack that fits the pruned set to a budget.

Long-context architectures. Efficient transformers (Tay et al., 2022) lower the marginal cost of a long resident context but carry the same content; they make the waste cheaper. The present discipline changes *what* is carried—residue, not knowledge—so it composes with, rather than competes with, cheaper-context machinery: a cheaper context makes the regenerated knowledge of step 5 cheaper still.

Anytime computation. The decline rule (theorem 9.4) is the anytime stance (Dean and Boddy, 1988; Zilberstein, 1996) with the stopping criterion moved from elapsed budget to non-resolution against the floor.

11.2 What the calculus assumes and does not supply

The one input the calculus does not derive is the term map τ—which distinctions a step draws—from which the context graph is built (§2). Everything downstream of τ is forced: given the graph, the floor, residue, necessity, the knapsack, the cascade, and the separation are all theorems. The map τ is a modelling choice, and its quality bounds the quality of the carry: a coarse τ (e.g. surface term overlap) yields a coarse but sound carry; a rich τ (e.g. a learned individuation of distinctions) yields a finer one. The calculus is agnostic to how τ is obtained and correct for any τ; sharpening τ is the natural axis of future work and the point at which a learned component, if desired, enters—bounded to the single task of drawing distinctions, with all necessity, allocation, and routing remaining deterministic.

11.3 Scope

The premises—finite agent, non-completable medium—hold for essentially every realised reasoning system with bounded memory and an open-ended task. Where an agent is effectively unbounded

(a perfect oracle) or its medium is completable (a closed, finite world it can exhaust), the floor may vanish and the discipline is not compelled: with $\beta = 0$, residues can reach zero, "solved" becomes a point, and there is no invariant to carry in preference to the knowledge. In that degenerate regime the standard carry is not wasteful, because there is nothing smaller than the knowledge to keep. The interest of the discipline is exactly that realised agents are *not* in that regime.

11.4 Conclusion

A finite agent reasoning over a growing history should not carry the history. It should carry the *residue*—the strictly positive, conserved, causal uncertainty each step deposits—and regenerate the *knowledge*—the relabellable, regenerable surface—on demand, and only the fraction each step needs. We derived the floor that makes residue positive and knowledge disposable, proved that necessity for a goal is reachability from it, that dropping the purposeless remainder is free, that the budgeted carry is an optimal knapsack over residue density, that repeated carries organise into a cascade with a logarithmic working set, and that relevance and necessity are distinct computations forcing a tandem of two operators. The discipline is deterministic, model-free given the context graph, and bounds the per-step context cost by the goal's reachable slice and a fixed budget rather than by the length of the history. The uncertainty is the state; the knowledge is the query answered anew each step. *Carry the uncertainty, not the knowledge.*

References

Max Black. Vagueness: An exercise in logical analysis. *Philosophy of Science*, 4(4):427–455, 1937.

George B. Dantzig. Discrete-variable extremum problems. *Operations Research*, 5(2):266–288, 1957.

Thomas Dean and Mark Boddy. An analysis of time-dependent planning. In *Proceedings of the Seventh AAAI National Conference on Artificial Intelligence*, pages 49–54, 1988.

Reinhard Diestel. *Graph Theory*. Springer, 5th edition, 2017.

L. R. Ford and D. R. Fulkerson. Maximal flow through a network. *Canadian Journal of Mathematics*, 8:399–404, 1956.

Hans Kellerer, Ulrich Pferschy, and David Pisinger. *Knapsack Problems*. Springer, 2004.

Leslie Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565, 1978.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems 33*, 2020.

Karl Menger. Zur allgemeinen kurventheorie. *Fundamenta Mathematicae*, 10:96–115, 1927.

George L. Nemhauser and Laurence A. Wolsey. *Integer and Combinatorial Optimization*. Wiley, 1988.

Sartaj Sahni. Approximate algorithms for the 0/1 knapsack problem. *Journal of the ACM*, 22 (1):115–124, 1975.

Herbert A. Simon. A behavioral model of rational choice. *The Quarterly Journal of Economics*, 69(1):99–118, 1955.

Herbert A. Simon. Rational choice and the structure of the environment. *Psychological Review*, 63(2):129–138, 1956.

Robert Tarjan. Depth-first search and linear graph algorithms. *SIAM Journal on Computing*, 1 (2):146–160, 1972.

Yi Tay, Mostafa Dehghani, Dara Bahri, and Donald Metzler. Efficient transformers: A survey. *ACM Computing Surveys*, 55(6):1–28, 2022.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30*, 2017.

Shlomo Zilberstein. Using anytime algorithms in intelligent systems. *AI Magazine*, 17(3):73–83, 1996.